

Q1:

It is recommended to explicitly assign values to enum members when we need fixed and meaningful values, especially if the values are stored in a database or used by another system.

This prevents problems if the enum members are reordered or new members are added later.

Q2:

It will cause a compile-time error because the assigned value is outside the range of the enum's underlying type.

Q3:

The virtual keyword allows a property to be overridden in a derived class, so the derived class can provide its own implementation.

Q4:

Because sealed prevents further overriding. Once a property or method is sealed, derived classes cannot change its implementation.

Q5:

Static members belong to the class itself, while object members belong to each individual object.

Static members are shared by all objects, but each object has its own copy of object members.

Q6:

No, not all operators can be overloaded in C#. Some operators, like `==`, `?:`, and `??`,

cannot be overloaded because their behavior is fixed by the language.

Q7:

You should consider changing the underlying type when you need a different range of values or want to reduce memory usage.

Q8:

Because a static class cannot be instantiated, so an instance constructor has no purpose.

Static classes can only have a static constructor.

Q9:

Enum.TryParse is safer because it doesn't throw an exception if the input is invalid.

It returns true if the conversion succeeds and false if it fails, making it easier to validate user input.

Q10:

Equals() is used to check whether two objects have the same value, while `==` normally checks reference equality for classes.

For structs, `==` compares values only if the operator is overloaded.

Q11:

Overriding ToString() makes it easier to display meaningful information about an object instead of the default class name.

Q12:

Yes

```
class Repository<T> where T : class
{
}
```

Q13:

A generic class uses a type parameter for the whole class,

while a generic method uses a type parameter only for a specific method.

A generic method can also be inside a non-generic class.

Q14:

Because a generic Swap method can work with different data types without writing a separate method for each type.

It reduces code duplication and makes the code more reusable.

Q15:

Overriding Equals() lets the Department class compare objects based on their actual data,

such as the department ID or name, instead of just comparing references.

This makes searches more accurate when looking for a specific department.

Q16:

Because C# doesn't know how you want to compare the fields of a custom struct.

So, `==` must be overloaded if you want to compare two struct objects using `==`.

## 2- Generalization using Generics:

It means writing one reusable piece of code that can work with different data types instead of creating separate code for each type.

## 3- Hierarchy design in real business:

It means organizing entities into levels based on their relationships, common features are placed in a general class and specific features are placed in derived classes.



Search

**Salma Mahmoud** · You

Computer Science Student at October 6 Univer...

2d · 

## Class Types in C#: More Than Just a Class

لما بدأت أتعلم C#, كلمة class كانت بالنسبالي معناها حاجة بسيطة:

نعمل Class → نعمل Object → نستخدمه.

لكن مع الوقت اكتشفت إن الموضوع مش بالبساطة دي.

في C#, الـ Class ممكن ياخد أشكال مختلفة، وكل شكل معمول لسبب معين.

وده خلاني أبص للـ Class مش كـ syntax بنكتبه، لكن كـ design choice بنحدد بيه إحنا عايزين الكود يتصرف إزاي.

### 1. Concrete Class

وده النوع الي غالبًا بنبدأ بيه.

هو الـ Class العادي الي نقدر نعمل منه Object مباشرة باستخدام .new

```
class Car
{
    public void Drive()
    {
        Console.WriteLine("Driving...");
    }
}

Car car = new Car();
car.Drive();
```

هنا الـ Class بيمثل حاجة فعلية نقدر نستخدمها وننشئ منها Objects.



Home



My Network



Post



Notifications



Jobs



Search



## 2. Abstract Class

هنا الموضوع مختلف شوية.

الـ Abstract Class مش بنعمل منه Object مباشرة.

فكرته إنه يكون base class فيه حاجات مشتركة بين Classes تانية، ويقدر يفرض عليهم behavior معين.

```
abstract class Animal
{
    public abstract void MakeSound();
}
class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Woof!");
    }
}
```

أنا مش محتاجة أعمل:

```
new Animal();
```

لأن Animal هنا مش المقصود يكون Object لوحده، هو بيمثل فكرة عامة، والـ derived classes هي اللي بتحدد التفاصيل.

وده كان من أكثر الحاجات اللي خلّتي أفهم إن الـ Class مش دايماً لازم يكون حاجة بنعمل منها Object.

## 3. Static Class

الـ Static Class مختلف تماماً.

هنا مفيش Objects أصلاً، وكل الـ members بتكون static.



Home



My Network



Post



Notifications



Jobs



Search



### 3. Static Class

الـ Static Class مختلف تمامًا.

هنا مفيش Objects أصلاً، وكل الـ members بتكون static.

```
static class Calculator
{
    public static int Add(int x, int y)
    {
        return x + y;
    }
}
int result = Calculator.Add(5, 3);
```

بنستخدمه لما يكون الـ Class نفسه هو اللي بنحتاجه، مش Instance منه.

### 4. Sealed Class

الـ Sealed Class بتقول ببساطة:

"استخدمني، بس متعملش inheritance مفي"

```
sealed class PaymentService
{
}
```

مينفعش بعد كده Class ثاني يعمل:

```
class NewPaymentService : PaymentService
{
}
```

وده بيكون مفيد لما نكون عايزين نمنع إن Class معين يتم توريثه.



Home



My Network



Post



Notifications



Jobs



Search



## 5. Partial Class

Partial Class مش بيغير طريقة إنشاء الـ Object، لكنه بيسمح لنا نقسم تعريف نفس الـ Class على أكثر من file.

```
public partial class Student
{
    public string Name { get; set; }
}
```

وفي file ثاني:

```
public partial class Student
{
    public void Study()
    {
        Console.WriteLine("Studying...");
    }
}
```

وفي النهاية، الـ Compiler بيجمع الأجزاء دي ويعاملهم كـ Class واحد.

وده بيكون مفيد خصوصًا في المشاريع الكبيرة أو مع الـ generated code.

## The important part

الموضوع مش إننا نحفظ أسماء:

Concrete — Abstract — Static — Sealed — Partial

الأهم إننا نفهم إمتى وليه نستخدم كل واحد.

لأن مع زيادة حجم المشروع، اختيار شكل الـ Class ممكن يفرق فـ الـ design، الـ inheritance، وتنظيم الكود



Home



My Network



Post



Notifications



Jobs



Search



The important part

الموضوع مش إننا نحفظ أسماء:

Concrete — Abstract — Static — Sealed — Partial

الأهم إننا نفهم إمتى وليه نستخدم كل واحد.

لأن مع زيادة حجم المشروع، اختيار شكل الـ Class ممكن يفرق في الـ design، الـ inheritance، وتنظيم الكود.

وده واحد من الحاجات اللي بدأت تخليني أشوف الـ OOP بشكل مختلف:

مش مجرد كتابة Objects و Classes...

لكن التفكير في إيه الـ design المناسب للمشكلة اللي بحاول أحلها.

And that's probably one of the most interesting parts of learning OOP —

the more you learn, the more you realize that writing code is only part of the job.

#CSharp #OOP #DotNet #Programming  
#SoftwareDevelopment #LearningJourney

Show translation



## Class Types in C#

More Than Just a Class

### 1 Concrete Class



Create objects

### 2 Abstract Class



Base for other classes

### 3 Static Class



No objects



Home



My Network



Post



Notifications



Jobs



Search



And that's probably one of the most interesting parts of learning OOP —

the more you learn, the more you realize that writing code is only part of the job.

#CSharp #OOP #DotNet #Programming  
#SoftwareDevelopment #LearningJourney

Show translation



## Class Types in C#

More Than Just a Class

### 1 Concrete Class



Create objects

```
class Car { }
var car = new Car();
```

### 2 Abstract Class



Base for other classes

```
abstract class Animal { }
class Dog : Animal { }
```

### 3 Static Class



No objects

```
static class Calculator { }
Calculator.Add(5, 3);
```

### 4 Sealed Class



No inheritance

```
sealed class PaymentService { }
```

### 5 Partial Class



Split across files

```
public partial class Student { }
```

Same keyword ... different purposes



Abdul Aziz Hamza and 13 others



Like



Comment



Repost



Send



176 impressions

View



Home



My Network



Post



Notifications



Jobs